

Multi-Source Candidate Data Transformer

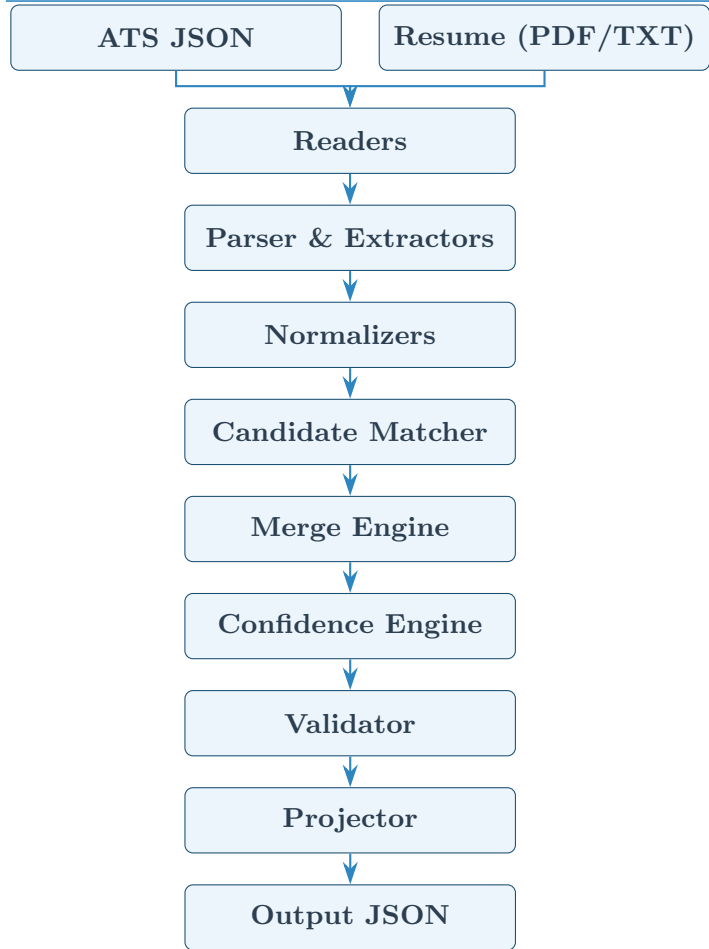
Vidish Kumar vidishkumar890@gmail.com github.com/Vidish3442

Eightfold Engineering Intern Assignment — Jul–Dec 2026

Objective

Design a configurable pipeline that transforms structured (ATS JSON) and unstructured (Resume PDF/TXT) candidate data into a canonical profile through extraction, normalization, conflict resolution, confidence scoring, and runtime-configurable output projection.

Pipeline Design



Each source is converted into a canonical representation before matching and merging. This separates source-specific parsing from business logic and improves maintainability.

Canonical Schema & Normalization

```
{ candidate_id, full_name, emails[ ], phones[ ], location{city, state, country}, links{ }, headline, years_experience, skills[ ], experience[ ], education[ ], certifications[ ], provenance[ ], overall_confidence }
```

Normalization Rules:

- Phones → E.164 format
- Dates → YYYY-MM
- Countries → ISO 3166-1 alpha-2
- Skills → Canonical aliases (e.g. py → Python)
- Lists → Deduplicated after normalization

Merge & Conflict Strategy

Matching: Exact email matching followed by normalized phone matching and fuzzy name comparison when required.

Scalar fields (name, email, phone): config-driven priority list — first non-empty value in the list wins. Default: `ats > resume`.

List fields (skills, experience, education):

- `Union + dedup` — ATS entries treated as canonical base.
- `Skills`: Fuzzy string matching canonicalizes aliases (e.g. "py" → "Python").
- `Experience`: using title and company similarity.
- `Education`: Institution-based matching with degree reconciliation.

Every merge decision writes to **Provenance Tracker** rather than constructing provenance inline, keeping merge logic and audit-trail construction fully decoupled.

Confidence Calculation

$$C_f = \min(0.99, W_{\max} + B_{\text{agree}} - P_{\text{norm}})$$

where:

- $W_{\max}$ = max source weight among contributing sources
- B_{agree} = 0.05 if ≥ 2 sources agree, else 0
- P_{norm} = 0.15 if normalization failed, else 0

Source weights: ATS = 0.95, Resume = 0.85, Recruiter = 0.70

Overall confidence = weighted average of all field confidences, weighted by field importance (w_f):

$$C_{\text{overall}} = \frac{\sum_f C_f \cdot w_f}{\sum_f w_f}$$

Runtime Config Handling

All output shaping lives in `config.yaml` — no code changes needed:

- `fields` — select which fields appear in transformed output
- `rename` — map canonical key to output key (`phone` → `mobile`)
- `include_confidence` / `include_provenance` — toggle meta-data
- `missing_policy` — `null` (keep as JSON null) — `omit` (drop key) — any string (default value)
- `source_priority` — per-field list for scalars; `{merge: union}` for list fields
- `matching.fuzzy_threshold` — rapidfuzz score threshold for candidate identity matching

Configuration is validated before execution. Projection applies field selection, renaming, and metadata policies while preserving the canonical internal representation.

Edge Cases Handled

- Scanned PDF → continue with ATS only
- Malformed ATS JSON → log warning and apply missing policy
- Invalid phone/date → keep raw value and reduce confidence
- Skill aliases (py, postgres, k8s) → canonical mapping
- Duplicate experience entries → merge descriptions

Out of Scope

- OCR support for scanned image resumes
- LinkedIn / GitHub crawling
- Multilingual resume parsing
- ML-based confidence estimation
- Batch processing of multiple candidates

Assumptions

- One ATS record + one resume per pipeline run (single-candidate mode)
- Resume is in English with standard section headers (EXPERIENCE, EDUCATION, SKILLS, etc.)
- ATS `candidate_id` is stable and used as the canonical primary key
- Source reliability weights are static (configured, not learned)
- Date strings are parseable by `dateparser` (e.g. "Jan 2023", "2023-01", "Present")
- Pipeline execution is deterministic for identical inputs.